BENDIS: Accurate and Hardware-Friendly Detection of Periodic Bursts in High-Speed Data Streams (Technical Report)

Weihe Li
University of Edinburgh
Edinburgh, UK
weihe.li@ed.ac.uk

Xicheng Li
University of Glasgow
Glasgow, UK
xicheng.li@glasgow.ac.uk

Dimitrios Pezaros
University of Glasgow
Glasgow, UK
dimitrios.pezaros@glasgow.ac.uk

Paul Patras
University of Edinburgh
Edinburgh, UK
paul.patras@ed.ac.uk

This report supplements the paper "BENDIS: Accurate and Hardware-Friendly Detection of Periodic Bursts in High-Speed Data Streams" with material that the paper omits for space. Section S1 proves the results of Section 4 and gives a closed form of the collision bound. Section S2 details the experimental setup, in particular the baselines. Section S3 details the Tofino and FPGA implementations. Section S4 reports results that the paper does not show. Sections, figures, tables, equations, Algorithm 1, Lemma 4.1, and Theorems 4.2–4.4 without the prefix S refer to the paper.

S1 Proofs of Section 4

We use the notation and the assumptions (A1)–(A3) of Section 4. All probabilities are over the hash functions; the stream, and thus every count $c_t(u)$, is fixed. We write C_r for the counter $B_r^t[h_r(e)]$ of e in row r and $\hat{c}^{(n)}$ for the value of $\hat{c}$ after the n-th arrival of e in window t, so that $\hat{c}_t(e) = \hat{c}^{(c_t(e))}$.

S1.1 Rotating Banks

LEMMA S1 (ROTATING BANKS). *Under (A2), for every row r and window t: (i) every counter of B_r^t is zero at the first arrival of window t; and (ii) during window t, B_r^{t-1} is not modified and holds its values at the end of window $t-1$.*

PROOF. All banks start at zero, so (i) holds for $t = 1$ and $t = 2$. In window t, Algorithm 1 increments only counters of B_r^t (Line 6), only reads B_r^{t-1} (Line 7), and zeroes counter z of B_r^{t+1} at the $(z+1)$-th arrival (Line 2). With $N_t \geq w$ arrivals, or with injected packets that complete the clearing, all w counters of B_r^{t+1} are thus zero at the end of window t, and B_r^{t+1} receives no increment in window t; this proves (i) for window $t+1$. In window $t+1$, B_r^t plays the role of the previous bank, which Algorithm 1 only reads, which proves (ii). □

S1.2 Per-Window Counts

LEMMA 4.1 (PAPER). *Under (A1)–(A3), for any item e and window $t \in \mathcal{A}(e)$, $c_t(e) \leq \hat{c}_t(e) \leq c_t(e) + \min_r X_{r,t}(e)$, and for every $\lambda > 0$, $\Pr[\hat{c}_t(e) - c_t(e) \geq \lambda] \leq \Pr[\min_r X_{r,t}(e) \geq \lambda] \leq \beta_t(\lambda)^d$, where $\beta_t(\lambda)$ is given by Eq. (3). The same bounds hold for $\hat{p}_t(e)$ with window $t-1$.*

PROOF. *Lower bound.* We show by induction over the arrivals of e in window t that, after its n-th arrival, $C_r \geq n$ for every row and $\hat{c}^{(n)} \geq n$. By Lemma S1(i), the counters start at zero, and counters never decrease within a window, so $C_r \geq n-1$ before the n-th arrival. Line 6 of Algorithm 1 sets $x = C_1 + 1 \geq n$ in row 1, as $\hat{c}$ starts at ∞. In row $r > 1$, if $C_r < \hat{c}$, then $x = C_r + 1 \geq n$; otherwise, $x = C_r \geq \hat{c} \geq n$ by the induction over the rows. Hence every updated counter, and their minimum $\hat{c}^{(n)}$, is at least n, and $\hat{c}_t(e) \geq c_t(e)$.

Upper bound. A counter starts the window at zero (Lemma S1(i)) and grows by at most one per arrival of window t that hashes to it, so $C_r \leq c_t(e) + X_{r,t}(e)$ at any time of window t. As $\hat{c}_t(e)$ is the minimum of the updated counters at the last arrival of e, $\hat{c}_t(e) \leq c_t(e) + \min_r X_{r,t}(e)$.

Tail. By the upper bound, $\Pr[\hat{c}_t(e) - c_t(e) \geq \lambda]$ is at most $\Pr[\min_r X_{r,t}(e) \geq \lambda]$. By (A1), $X_{r,t}(e)$ depends only on h_r, so the rows are independent and $\Pr[\min_r X_{r,t}(e) \geq \lambda] = \prod_r \Pr[X_{r,t}(e) \geq \lambda]$. It remains to show that $\Pr[X_{r,t}(e) \geq \lambda]$ is at most each term of $\beta_t(\lambda)$. The first term is trivial. For the second, $\mathbb{E}[X_{r,t}(e)] = \sum_{u \neq e} c_t(u)/w \leq N_t/w$, as $\Pr[h_r(u) = h_r(e)] = 1/w$ (pairwise independence suffices), and Markov's inequality gives $N_t/(w\lambda)$. For the third, fix $\theta \geq 1$. If $X_{r,t}(e) \geq \lambda$, then either a heavy item $u \in \mathcal{H}_t(\theta) \setminus \{e\}$ collides with e, which happens with probability at most $|\mathcal{H}_t(\theta)|/w$ by the union bound, or the light collisions $X^{\mathcal{L}} = \sum_{u \in \mathcal{L}_t(\theta) \setminus \{e\}} c_t(u) I_r(u)$ reach λ. Conditioned on $h_r(e)$, the indicators $I_r(u)$, $u \neq e$, are independent Bernoulli variables with mean $1/w$ under (A1), so for every $s > 0$, Markov's inequality applied to $e^{sX^{\mathcal{L}}}$ gives

$$\Pr[X^{\mathcal{L}} \geq \lambda] \leq e^{-s\lambda} \prod_{u \in \mathcal{L}_t(\theta) \setminus \{e\}} \left(1 + \frac{e^{s\,c_t(u)} - 1}{w}\right) \leq e^{\Psi_t(s;\theta) - s\lambda}, \tag{S1}$$

where the last step adds the nonnegative term of e if e is light. Taking the infimum over s and θ yields the third term.

Previous window. By Lemma S1(ii), $\hat{p}_t(e)$ is the minimum over the rows of the counters of e at the end of window $t-1$. The arguments above, applied at the end of window $t-1$, give $c_{t-1}(e) \leq \hat{p}_t(e) \leq c_{t-1}(e) + \min_r X_{r,t-1}(e)$ and the tail bound with β_{t-1}; if e has no arrival in window $t-1$, the lower bound is trivial. By (A3), a window carries fewer than 2^{16} arrivals, so no 16-bit counter saturates, and these bounds are exact statements about Algorithm 1. □

S1.3 A Closed Form

COROLLARY S2 (CLOSED FORM). *For every $\theta \geq 1$ and $\lambda > \nu_t(\theta)$, where $\nu_t(\theta) = \sum_{u \in \mathcal{L}_t(\theta)} c_t(u)/w$ bounds the mean of the light collisions,*

$$\beta_t(\lambda) \leq \frac{|\mathcal{H}_t(\theta)|}{w} + e^{-\nu_t(\theta)/\theta} \left(\frac{e\,\nu_t(\theta)}{\lambda}\right)^{\lambda/\theta} \leq \frac{|\mathcal{H}_t(\theta)|}{w} + \left(\frac{e\,N_t}{w\lambda}\right)^{\lambda/\theta}. \tag{S2}$$

PROOF. A light item has $c_t(u) \in [0, \theta]$, and by convexity, $e^{sc} - 1 \leq (c/\theta)(e^{s\theta} - 1)$ on this interval. With $\ln(1+x) \leq x$, Eq. (2) gives $\Psi_t(s;\theta) \leq (\nu_t(\theta)/\theta)(e^{s\theta} - 1)$. Setting $s = \ln(\lambda/\nu_t(\theta))/\theta$,

which is positive as $\lambda > \nu_t(\theta)$, yields $e^{\Psi_t(s;\theta)-s\lambda} \le \exp\big((\lambda - \nu_t(\theta))/\theta\big)\,(\nu_t(\theta)/\lambda)^{\lambda/\theta}$, i.e., the first bound. The second follows from $e^{-\nu_t(\theta)/\theta} \le 1$, $\nu_t(\theta) \le N_t/w$, and the monotonicity of $x \mapsto (ex/\lambda)^{\lambda/\theta}$. If $\nu_t(\theta) = 0$, all light counts are zero, $\Psi_t \equiv 0$, and letting $s \to \infty$ shows that the light term vanishes, which the first bound expresses with $0^{\lambda/\theta} = 0$; the second bound then holds a fortiori. □

For $\theta < \lambda$, the light term of Eq. (S2) vanishes as $w^{-\lambda/\theta}$, faster than the w^{-1} of Markov's inequality, so $\beta_t(\lambda) \le |\mathcal{H}_t(\theta)|/w + O(w^{-\lambda/\theta})$. At the memory sizes of our evaluation, however, the closed form is loose. With 32 KB ($w = 1{,}274$) and $\lambda = 64$ on CAIDA 2019, Markov's term is 0.368 in every window, the first bound of Eq. (S2), minimized over the values of θ of our grid (below), is 0.250 on average over the windows, and the second bound exceeds one, whereas Eq. (3) evaluated on the count profiles gives 0.119. The paper therefore evaluates Eq. (3) numerically on every count profile, with θ in $\{2, 3, 4, 6, \ldots, 768, 1024\}$, i.e., 2^j and $3 \cdot 2^{j-1}$, and $\theta = \infty$, and with $s = 10^{-4} \cdot 1.15^i < 40$ for integers $i \ge 0$; restricting the infima to grids can only loosen the bound.

S1.4 Burst-Start Errors

Theorem 4.2 (paper). *Under (A1)–(A3), if e has a burst start in window t with margin $\Delta_t = c_t(e)/k - c_{t-1}(e)$,* BendIS *misses it with probability at most $\beta_{t-1}(\lfloor\Delta_t\rfloor + 1)^d$. If $t \in \mathcal{A}(e) \setminus \mathcal{T}(e)$ with gap $\Gamma_t = \max(H, k\,c_{t-1}(e)) - c_t(e)$,* BendIS *detects a false start with probability at most $\beta_t(\Gamma_t)^d$. Hence, with $\beta_0 \equiv 0$, $\mathbb{E}\big[|\mathcal{M}(e)| + |\mathcal{F}(e)|\big]$ is at most $\mu(e)$ of Eq. (4).*

Proof. Within window t, $\hat{p}$ is constant by Lemma S1(ii), and $\hat{c}^{(n)}$ is nondecreasing in n: the counters only grow, and the updated counter of each row at a later arrival is at least its value after an earlier one. Hence some arrival of e passes the start test $\hat{c} \ge \max(H, k\hat{p})$ if and only if its last arrival does, i.e., if and only if $\hat{c}_t(e) \ge \max(H, k\hat{p}_t(e))$.

Missed starts. Let $t \in \mathcal{T}(e)$, i.e., $c_t(e) \ge H$ and $c_t(e) \ge k\,c_{t-1}(e)$; as $H \ge 1$, $t \in \mathcal{A}(e)$. By Lemma 4.1, $\hat{c}_t(e) \ge c_t(e) \ge H$, so the test fails only if $k\hat{p}_t(e) > \hat{c}_t(e) \ge c_t(e)$, i.e., $\hat{p}_t(e) - c_{t-1}(e) > c_t(e)/k - c_{t-1}(e) = \Delta_t$. As counts are integers, this needs $\hat{p}_t(e) - c_{t-1}(e) \ge \lfloor\Delta_t\rfloor + 1$, which has probability at most $\beta_{t-1}(\lfloor\Delta_t\rfloor + 1)^d$ by Lemma 4.1. For $t = 1$, $\hat{p}$ reads a bank that is still zero, so the burst start is never missed, which $\beta_0 \equiv 0$ expresses.

False starts. Let $t \in \mathcal{A}(e) \setminus \mathcal{T}(e)$, so $\Gamma_t = \max(H, k\,c_{t-1}(e)) - c_t(e) > 0$. A false start needs $\hat{c}_t(e) \ge \max(H, k\hat{p}_t(e))$. As $\hat{p}_t(e) \ge c_{t-1}(e)$, this implies $\hat{c}_t(e) \ge \max(H, k\,c_{t-1}(e))$, i.e., $\hat{c}_t(e) - c_t(e) \ge \Gamma_t$, which has probability at most $\beta_t(\Gamma_t)^d$. Windows without arrivals of e cannot yield a detected start.

Summing the two kinds of windows and using the linearity of expectation gives $\mathbb{E}[|\mathcal{M}(e)| + |\mathcal{F}(e)|] \le \mu(e)$. □

S1.5 Interval Retention

Theorem 4.3 (paper). *Under (A1)–(A3) and conditioned on the counting hashes, let the detected starts of e be exactly its burst starts and at most V windows apart, t_0 its first burst start, Q_t the number of other items with detected starts in windows $t - V, \ldots, t$, and U the number of other items with detected starts in windows $\min \mathcal{A}(e) - V, \ldots, \max \mathcal{A}(e)$. Then all intervals of e are reported exactly with probability at least $1 - Q_{t_0}/b - U/(b\,2^\ell)$.*

Proof. Conditioned on the counting hashes, the detected starts of all items are fixed, and by (A1), the table hashes $g_1, \ldots, g_m$ and the fingerprint f are independent of them; we only credit row 1. An item writes an entry only at its detected starts (Lines 10–12), so an entry is fresh in window t only if an item with a detected start in windows $t - V, \ldots, t$ wrote it.

First burst start. At t_0, e probes $E = O_1[g_1(e)]$. If no other item with a detected start in windows $t_0 - V, \ldots, t_0$ hashes to this entry under g_1, then E is stale, and e claims it; this burst start reports no interval, as $v > V$, and none exists. By the union bound over these Q_{t_0} items, each hashing to the entry with probability $1/b$, the claim fails with probability at most Q_{t_0}/b.

Later starts. Once e holds E, consecutive starts of e are at most V windows apart, so E stays fresh, and another item u can overwrite it only through a fingerprint match, i.e., if $g_1(u) = g_1(e)$ and $f(u) = f(e)$, which happens with probability $1/(b\,2^\ell)$. Such a u has a detected start between t_0 and the last start of e, i.e., within the windows of U, so by the union bound, no such overwrite occurs with probability at least $1 - U/(b\,2^\ell)$. Then every later burst start of e in window t matches E, whose window τ is the previous burst start of e, and Lines 12–13 report $v = t - \tau$ if $R < v \le V$, i.e., exactly the intervals of e; intervals of at most R windows are not counted in the ground truth either, and τ advances to the latest burst start in both. Later arrivals of e in a burst-start window give $v = 0$ and are not reported. The union of the two failure events completes the proof. □

S1.6 Frequency Error

Theorem 4.4 (paper). *Under (A1)–(A3), let the burst starts of e be at most V windows apart, with t_0, Q_t, and U as in Theorem 4.3, expectations over the counting hashes, and $\bar{Q} = \mathbb{E}[\max_{t \in \mathcal{A}(e)} Q_t]$. For every bin $\bar{v}$ and $\epsilon > 0$,*

$$\Pr\left[|\hat{n}(e,\bar{v}) - n(e,\bar{v})| \ge \epsilon\right] \le \frac{2\mu(e)}{\epsilon} + \left\lceil\frac{\epsilon}{2}\right\rceil\frac{\bar{Q}}{b} + \frac{\mathbb{E}[U]}{b\,2^\ell},$$

i.e., Eq. (5), and all frequencies of e are exact with probability at least $1 - \mu(e) - \mathbb{E}[Q_{t_0}]/b - \mathbb{E}[U]/(b\,2^\ell)$.

Proof. Let $D = |\mathcal{M}(e)| + |\mathcal{F}(e)|$. For a set S of windows, let $\phi(S, \bar{v})$ be the number of pairs of consecutive windows of S whose gap g satisfies $R < g \le V$ and $\lfloor g/\delta\rfloor = \bar{v}$. As the burst starts of e are at most V windows apart, $n(e,\bar{v}) = \phi(\mathcal{T}(e), \bar{v})$. We say that the table *works* if the reports of e are exactly the gaps g with $R < g \le V$ between its consecutive detected starts, so that $\hat{n}(e,\bar{v}) = \phi(\hat{\mathcal{T}}(e), \bar{v})$.

Burst-start errors. Deleting a window from S removes the gaps to its neighbors in S, at most two, and adds their sum if it has two neighbors, so it changes every $\phi(S, \bar{v})$ by -2 to $+1$; inserting a window changes it by -1 to $+2$. Deleting the $|\mathcal{F}(e)|$ false starts from $\hat{\mathcal{T}}(e)$ and inserting the $|\mathcal{M}(e)|$ missed ones yields $\mathcal{T}(e)$, so if the table works, $|\hat{n}(e,\bar{v}) - n(e,\bar{v})| \le 2D$.

Table failures. Conditioned on the counting hashes, the detected starts of all items are fixed, and as in the proof of Theorem 4.3, e keeps its row-1 entry between two detected starts at most V windows apart unless a fingerprint match overwrites it, and its matches then report exactly the gaps between its consecutive detected starts that lie in $(R, V]$; a detected start more than V windows after the previous one reports nothing, whether e still holds its entry or

claims one. Hence e must claim an entry (a *placement*) only at its first detected start and at each detected start whose predecessor is more than V windows earlier, and the table works unless a placement fails or a fingerprint match overwrites the entry of e. If e has no burst start, every detected start is false, and there are at most D placements. Otherwise, we charge every gap of more than V windows between consecutive detected starts $a < a'$ to a distinct burst-start error. If a burst start of e lies strictly between a and a', it is missed, and we charge it; each missed start lies in a single gap. Otherwise, as consecutive starts of e are at most V windows apart, a precedes the first burst start of e or a' follows its last one, so a or a', respectively, is a false start, which we charge; a false start before the first burst start is charged only for the gap that starts at it, and one after the last start only for the gap that ends at it. Hence there are at most $1 + D$ placements. A placement in window $t \in \mathcal{A}(e)$ fails only if one of the Q_t other items with detected starts in windows $t - V, \ldots, t$ hashes to the same row-1 entry, which has probability at most Q_t/b, and a fingerprint match with one of at most U items has probability at most $U/(b\,2^\ell)$.

Combining. If $|\hat{n}(e,\bar{v}) - n(e,\bar{v})| \geq \epsilon$, then $D \geq \epsilon/2$, or $D < \epsilon/2$ and the table fails, in which case there are at most $1 + D \leq \lceil \epsilon/2 \rceil$ placements, as D is an integer. By Markov's inequality and Theorem 4.2, $\Pr[D \geq \epsilon/2] \leq 2\mu(e)/\epsilon$, and conditioned on the counting hashes, the second event has probability at most $\lceil \epsilon/2 \rceil \max_{t \in \mathcal{A}(e)} Q_t/b + U/(b\,2^\ell)$; taking the expectation over the counting hashes gives Eq. (5). For exactness, if e has no burst start, $D = 0$ means that it has no detected start, so its frequencies are exact with probability at least $1 - \mu(e)$; otherwise, the frequencies are exact if $D = 0$ and the table works. If $D = 0$, the detected starts of e are its burst starts, so its only placement is at t_0, and conditioned on the counting hashes, the table fails with probability at most $Q_{t_0}/b + U/(b\,2^\ell)$. Hence the frequencies are wrong with probability at most $\Pr[D \geq 1] + \mathbb{E}\left[\mathbf{1}[D = 0](Q_{t_0}/b + U/(b\,2^\ell))\right] \leq \mu(e) + \mathbb{E}[Q_{t_0}]/b + \mathbb{E}[U]/(b\,2^\ell)$, using $\Pr[D \geq 1] \leq \mathbb{E}[D] \leq \mu(e)$. □

S1.7 Memory Provisioning

Consider an item whose burst starts are at most V windows apart, with n_a active windows, half of them burst starts with $\Delta_t \geq \Lambda$ and half other windows with $\Gamma_t \geq \Lambda$, for an integer Λ. As β_t is nonincreasing in λ and $\lfloor \Delta_t \rfloor + 1 \geq \Lambda + 1$, $\beta_{t-1}(\lfloor \Delta_t \rfloor + 1) \leq \max_t \beta_t(\Lambda + 1)$ and $\beta_t(\Gamma_t) \leq \max_t \beta_t(\Lambda)$, so Eq. (4) gives $\mu(e) \leq \frac{n_a}{2}\left(\max_t \beta_t(\Lambda + 1)^d + \max_t \beta_t(\Lambda)^d\right)$, which the first condition of Eq. (6) bounds by $\eta/3$. With $Q^* \geq \mathbb{E}[Q_{t_0}]$ and $U^* \geq \mathbb{E}[U]$, the second condition gives $\mathbb{E}[Q_{t_0}]/b + \mathbb{E}[U]/(b\,2^\ell) \leq 2\eta/3$, so by Theorem 4.4, the frequencies of the item are exact with probability at least $1 - \eta$. For any K items fixed independently of the hash functions, e.g., those of the true top-K pairs, the union bound replaces η by η/K; the reported items depend on the hashes, so the union bound does not apply to them directly. With Markov's term alone, the first condition reads $\frac{n_a}{2}\left((N_t/(w(\Lambda + 1)))^d + (N_t/(w\Lambda))^d\right) \leq \eta/3$, which gives w_{M} in closed form. Every term of β_t is nonincreasing in w, as $\ln(1 + x/w)$ decreases with w for $x > 0$, so the first condition, once it holds, holds for every larger w, and w_{C} is found by a binary search over w that evaluates Eq. (3) on every count profile. The example of Eq. (7) uses $W = 30{,}000$, estimates Q^* and U^* by the largest Q_t and the number of all items with detected starts over three seeds with 64 KB, which has fewer counters than w_{C} and thus more detected starts than the provisioned sketch, and provisions the second table row without crediting it, as in Theorems 4.3 and 4.4.

S2 Experimental Setup Details

► **Platform.** The CPU of Section 6.1 is a KVM virtual machine. Throughput runs are pinned to one core with nothing else running, and the throughput of a run is the number of arrivals divided by the time to process them and to aggregate the reports. A hash seed changes all hash functions of every method; the comparison with the hardware implementations and the control-plane load (Tables S3 and S4) use a single run, as the CRC hashes of the switch are fixed.

► **Ground Truth and Metrics.** The ground truth counts every item in every window exactly and finds its burst starts (Section 2.1). A top-K list, for the ground truth and for the reports of every method alike, takes the pairs in descending order of frequency and stops at the first pair whose frequency is below that of the $(K-1)$-th pair, as the comparer of the released code of PBSketch [1] does. A reported pair is correct if it belongs to the ground-truth list; precision and recall are the fractions of correct pairs among the reported pairs and among the ground-truth pairs. Counting a missed pair with the estimate 0 in the ARE keeps a method from lowering its error by reporting fewer pairs. A value of K is evaluated on a stream only if the K-th ground-truth pair recurs at least twice; otherwise, ties at frequency one extend the list to almost every pair.

► **PBSketch.** We build PBSketch from its released code [1]. Besides the hot cell, a bucket of its first part has cold cells with fingerprints and counts of other items; its second part keeps the most frequent pairs. The start-reporting version gives all memory to the first part. A test checks that, with the reporting of burst starts disabled, our wrapper reproduces the reports of the first part of the released code window by window. The *original version* (Fig. 9(f)–(j)) is the released code with its own definition and memory split (80% for the first part). The released code also contains PBSketchOpt, which processes a bucket lazily at its next access instead of at window boundaries. On CAIDA 2019, it processes 26.1 Mops on average over the memory sizes, versus 30.5 Mops for the start-reporting version and 37.3 Mops for Bendis in the same interleaved runs, so the start-reporting version is the faster baseline.

► **Configurations of PBSketch.** The released code stores the two window numbers of a bucket with $\lceil \log_2 16 \rceil = 4$ bits each but charges them as 8 bytes per bucket, as much as two 32-bit fields; with 4 bits, intervals of 16 windows or more wrap around. We therefore consider three configurations: as released, with 4-bit window numbers charged at 8 bits per bucket, and with 7-bit window numbers charged at 14 bits per bucket, in each case filling the memory budget with buckets. For every stream, memory size, K, and definition, we report the configuration with the highest mean F1 score over the seeds, and for the ARE (Fig. 9(a)–(e)), the one with the lowest mean ARE. Exact accounting gives PBSketch more buckets than the released code for the same memory, so the experiments that we ran after adopting it, i.e., the Twitter stream, the ARE, and the stream length (Fig. 12(g)), use only the two exactly charged configurations. Throughput (Fig. 10) uses 4-bit window

numbers charged exactly. For Fig. 12(h), the cold cells store 8-byte keys instead of 16-bit fingerprints.

► **BurstSketch+PeriodicSketch.** We configure the chained baseline as in the benchmark of PBSketch [1]: BurstSketch [3] takes 40% of the memory and forwards every completed burst to PeriodicSketch [2], which takes 60% and keeps the top intervals.

► **Memory Accounting.** For PBSketch, the memory budget covers its buckets with the window numbers charged as above, and for the chained baseline, its two sketches, which also hold its reported pairs. The exact aggregation of the reports of Bendis and of the start-reporting version of PBSketch in host memory is not charged (Sections 4 and 6.1).

► **Synthetic Streams.** The generator (`src/gen_synth.cpp`) fills every window of 30,000 arrivals with the planted items and draws the rest from 1M background items with a Zipf distribution. Every planted item has a base rate drawn uniformly from 0 to 3 arrivals per window, and all its counts are Poisson-distributed with the rate of the window. Each of the 1,000 periodic items has a period p drawn uniformly from the given range and an amplitude drawn uniformly from 90 to 200 arrivals, and bursts in one window every p windows, with the rate of the burst window set to the amplitude times a factor drawn from $[0.8, 1.2]$; 30% of the items with $p \geq 7$ add a tail that decays by a factor of 0.6 per window over three windows, a burst starts one window early or late with probability 0.3 and is skipped with probability 0.05. Each of the 150 aperiodic items bursts for one to four windows with an amplitude of 100 to 400 arrivals, scaled per window likewise, its bursts starting on average every 45 windows (geometric gaps). Each of the 20 level-shift items raises its rate to 100–300 arrivals per window for one to three periods of 50 to 300 windows.

► **Period Inference (Section 7.1).** For an item with burst starts $o_1 < o_2 < \ldots$ (burst starts within R windows of the previous one merged), the hit rate $h(L)$ of a lag L is the fraction of burst starts o_i that have another burst start within $L \pm \max(2, 0.03L)$ windows after them, and the score of L is $\min(h(L), h(2L), h(3L), h(4L))$, which is high for the period and low for the spacing of several bursts within one iteration. A lag is significant if its score is at least 0.6 and exceeds by 0.15 the 95th percentile of the scores of random burst-start sequences with the same number of burst starts and the same minimum gap. The period is the smallest significant lag whose score is within 5% of the best, replaced by a significant lag near its half, third, or quarter that scores at least 85% as high, and refined to the mean distance to the burst starts matched within 1%; every method uses this inference. The recall counts the 1,354 training connections in which exact counting finds burst starts in at least half of the iterations; the other 240 of the 1,594 training connections rarely or never burst by our definition, even under exact counting, so no detector can find their periods, and periods inferred for them are not scored.

► **Interleaving (Section 7.1).** The simulation drops each burst start with the miss rate of Bendis on the replayed switch but does not inject its false starts, which are 0.05% and 0.10% of its reports there with 32 and 16 KB.

Table S1: Stage placement of Bendis on Tofino 1 (`bf-p4c 9.13.4`, $w = b = 2^{11}$); the other sizes use the same placement.

Stage	Work
0	Arrival counter (clearing index z), window number, forwarding
1	Bank rotation register; CRC hashes h_1, h_2, h_3, g_1, g_2 and fingerprint f
2	Counting row 1: increment bank t, read bank $t-1$, and zero entry z of bank $t+1$
3	Counting row 2: increment iff the counter is below $\hat{c}$ of row 1
4	Counting row 3: increment iff the counter is below $\hat{c}$ of rows 1–2; minimum of rows 1–2
5–7	$\hat{c}$ and $\hat{p}$, $\lfloor \hat{c}/k \rfloor$ (shift), saturating differences for the start test
8	Start-table row 1 (only on a burst start)
9	Start-table row 2 (only if row 1 passes); interval from row 1
10	Interval from row 2
11	Digest iff $R < v \leq V$

S3 Implementation Details

S3.1 Tofino

Table S1 lists the stages that the compiler assigns. With the parameters of the warehouse use case (Section 7.2: $H = 1$, $R = 2$, and $V = 2{,}050$, i.e., $V - R = 2^{11}$), the program occupies the same 12 stages with the same SRAM, stateful ALUs, and hash bits, and the wider bit test of the report condition raises the gateway usage from 19.8% to 20.3%.

► **Hardware Constraints.** Beyond the single access per register array, Tofino imposes several constraints, each of which shaped the update procedure.

- *One output per stateful ALU.* A register action returns a single value, so a table row returns the old window τ when it writes and zero when it passes. Entries are initialized to window 1, which no valid window number takes, so claiming an unused entry yields an interval larger than V, which is not reported.
- *Two operands per stateful ALU.* A table row compares the stored fingerprint with $f(e)$ and the age $t - \tau$ with V, which uses exactly the two operands $f(e)$ and t; the conservative update of row 3 compares its counter with the outputs of rows 1 and 2.
- *Gateway width.* A gateway compares at most four bytes plus 12 bits, hence the zero tests of the start test and the bit test of the report condition (Section 5).
- *Hash output per action.* An action moves at most 32 bits of hash output, so each of the six hashes is computed by its own action.

► **Windows.** With time windows, a window spans 2^{23} ns (≈8.4 ms) by default, and its 16-bit number is taken from the ingress timestamp through an identity hash, which avoids realigning timestamp bits. With $V = 65$, windows of 2^{23} ns cover periods of up to 0.55 s and windows of 2^{26} ns up to 4.4 s. With count windows, used for trace replays, a register advances the window every 2^{15} packets by default and every 2^{12} packets in the replay of Section 6.5, and a replay spans at most 2^{15} windows. With time windows, window numbers carry a flag bit and wrap every 2^{15} windows, while the staleness test subtracts 16-bit values, so the control plane scans the start table periodically, every 60 s by default, i.e., more than V and at most 2^{14} windows, and resets every entry that did not change since its previous scan to the unused window 1, which stays stale.

Such an entry is older than V, so the reset changes no decision, and every untouched entry is reset within two scans; an entry thus never looks fresh again after a wrap. An interval is lost only if it spans a wrap or if its item updates the entry between the read and the write of a scan, which resets the entry and loses at most one interval. Counters saturate instead of wrapping.

► **Control Plane and Testbed.** The control plane resets stale entries as described above; a PTF test on the Tofino model checks that a reset entry makes the next start of its item a first burst start. We take the resource usage from the resource report of `bf-p4c` and validate the program on Intel's Tofino model under the switch driver `bf_switchd`: a PTF test sends the packets through virtual Ethernet interfaces and collects the digests, which we compare with those of the behavioral model. As the model does not reproduce the timing of the trace, the replay uses count windows.

S3.2 FPGA

In the 11-stage FPGA pipeline (Fig. 6), stage 0 derives the window, the clearing index (the low bits of the arrival counter), and the rotation, and the stage after the CRC hashes, which are generated as XOR matrices, issues the first block-RAM read. Every bank and table row is a simple dual-port block RAM accessed in a single stage; the forwarding selects for the writes of the two preceding arrivals are computed one cycle in advance. Windows hold 2^{WIN_BITS} arrivals, and window numbers wrap modulo 2^{15}, as with the time windows of the Tofino program; a host must then reset stale entries periodically, which our prototype does not include, as our simulations stream fewer than 2^{15} windows.

We synthesize the design with Yosys and place and route it with nextpnr for a Lattice ECP5 LFE5U-85F (CABGA756 package, speed grade 6, the slowest), with a 90 MHz timing target and a fixed placement seed. The resources and clock frequencies of Table 4, which may fall slightly below the target, are taken from the post-route reports. A cycle-accurate Verilator simulation of the RTL streams one arrival per cycle and runs in lockstep with the behavioral model of Section 5, checking that the report sequences are identical; we do not run the design on a physical board.

S4 Additional Results

This section reports results that the paper does not show. Unless stated otherwise, the settings follow Section 6.1.

S4.1 Larger K

The F1 score of a top-K list is meaningful only if the K-th ground-truth pair recurs at least twice (Section S2). On CAIDA 2016, only 791 pairs do, so the paper evaluates it up to $K = 500$. Figure S1 extends Fig. 7 to K from 800 to 1,500 on CAIDA 2018 and 2019, Zipf 1.0, and Periods 3–60. On average over these streams, values of K, and memory sizes, the F1 score of Bendis exceeds that of PBSketch by 0.27; with $K = 1{,}500$ and 32 KB, Bendis keeps an F1 score of at least 0.94, whereas PBSketch attains 0.46–0.64. The figure omits the Twitter stream, whose 1,500-th pair occurs only once; with $K = 800$ and 1,000, which both give its 1,276 pairs of frequency two or more, Bendis exceeds PBSketch by 0.37 on average over the memory sizes.

Table S2: Tofino 1 resources of Bendis that vary with w, averaged over the 12 stages (`bf-p4c 9.13.4`); the others are as in Table 3.

w	SRAM	Map RAM	Hash bits
2^9	2.81%	4.51%	3.19%
2^{10}	2.81%	4.51%	3.45%
2^{11}	2.81%	4.51%	3.71%
2^{12}	2.81%	4.51%	3.97%
2^{13}	3.02%	4.86%	4.23%

S4.2 Parameters on the Other CAIDA Traces

Figure S2 repeats Fig. 12(c)–(e), which use CAIDA 2019, on CAIDA 2016 and 2018. On average over the two traces and all values of H, W, and k, the F1 score of Bendis exceeds that of PBSketch by 0.10, and Bendis keeps at least 0.83.

S4.3 Design Parameters with 32 KB

Figure S3 repeats Fig. 13, which uses 16 KB, with 32 KB. The F1 score varies by at most 0.03 for a start-table share of 20–40% and by at most 0.03 for V from 33 to 129; a single counting row lowers it by 0.10–0.16, and a second table row adds up to 0.05. The defaults are within 0.03 of the best value of every parameter on every trace.

S4.4 Synthetic Streams at Every Memory Size

Figure S4 extends Fig. 12(a), (b), and (h), which use 32 KB, to every memory size; Fig. 7 already shows $\alpha = 1.0$ and periods of 3–60 windows. Bendis attains an F1 score of at least 0.953 for every skewness and memory size, whereas PBSketch attains 0.78–0.81 with 16 KB. The cost of 8-byte keys for Bendis shrinks as memory grows, from 0.16 with 16 KB to at most 0.01 from 64 KB on, and with 8-byte keys, Bendis stays ahead of PBSketch with fingerprints at every memory size.

S4.5 Hardware

Table S2 lists the Tofino resources that vary with w, which Table 3 summarizes. Table S3 lists the F1 scores of the CPU and hardware implementations for every K and w, of which Section 6.5 summarizes $K = 200$ to 1,000; they differ only through the hash functions, CRC on the hardware and seeded 64-bit hashes on the CPU. Table S4 details the control-plane load per trace and w: the digest rate barely depends on w, as it is set by the number of reported burst starts. For the FPGA, a Yosys synthesis of the same design with $w = 2^{11}$ for a Xilinx UltraScale+ device uses 3,288 LUTs, 1,361 flip-flops, and 13 36-Kb block RAMs, an estimate before place and route.

References

[1] Zhuochen Fan, Zhongxian Liang, Zirui Liu, Dayu Wang, Dong Wen, Wenjun Li, Tong Yang, Yuzhou Liu, and Weizhe Zhang. 2026. PBSketch: Source Code. https://github.com/wenjunpaper/PBSketch. Accessed October 2026.

[2] Zhuochen Fan, Yinda Zhang, Tong Yang, Mingyi Yan, Gang Wen, Yuhan Wu, Hongze Li, and Bin Cui. 2022. PeriodicSketch: Finding Periodic Items in Data Streams. In *Proc. IEEE ICDE*. 96–109.

[3] Zheng Zhong, Shen Yan, Zikun Li, Decheng Tan, Tong Yang, and Bin Cui. 2021. BurstSketch: Finding Bursts in Data Streams. In *Proc. ACM SIGMOD*. 2375–2383.

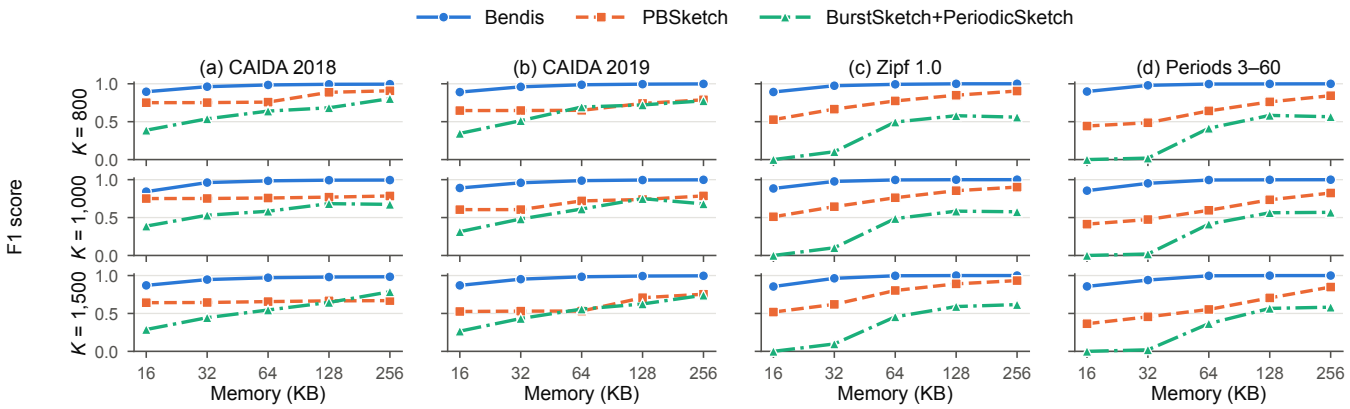


Figure S1: F1 score versus memory for $K = 800$, 1,000, and 1,500 (rows) on CAIDA 2018 and 2019 and on the synthetic streams Zipf 1.0 and Periods 3–60 (columns).

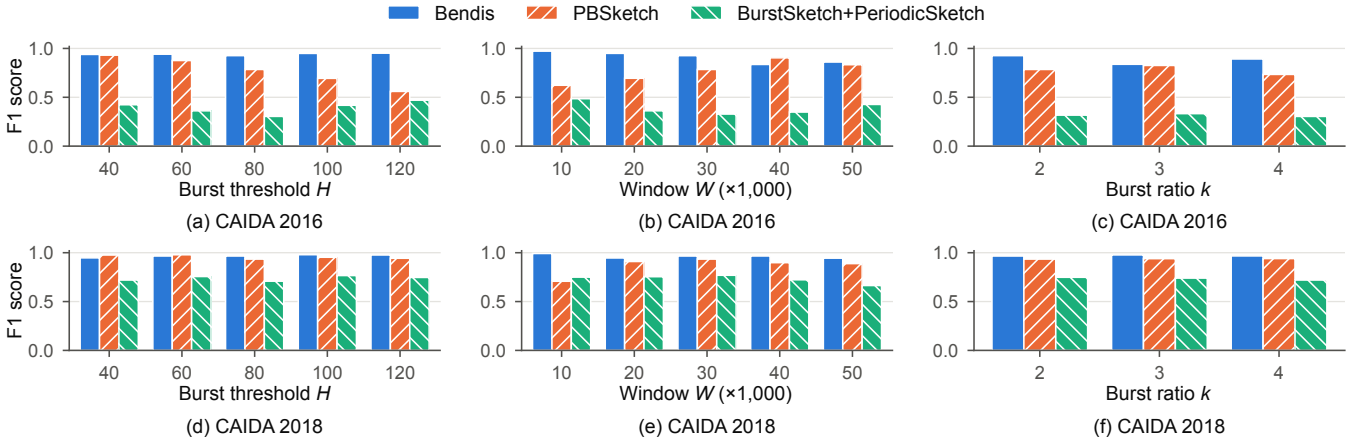


Figure S2: F1 score versus the burst threshold H, the window size W, and the burst ratio k on CAIDA 2016 and 2018 (32 KB, $K = 200$).

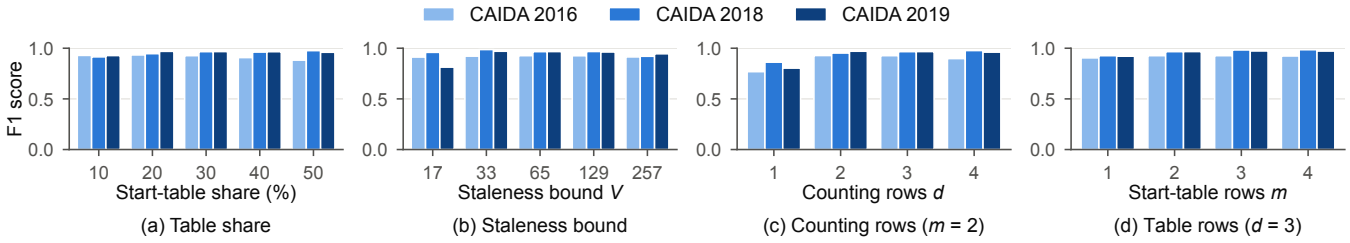


Figure S3: Design parameters of BENDIS on the CAIDA traces with 32 KB ($K = 200$).

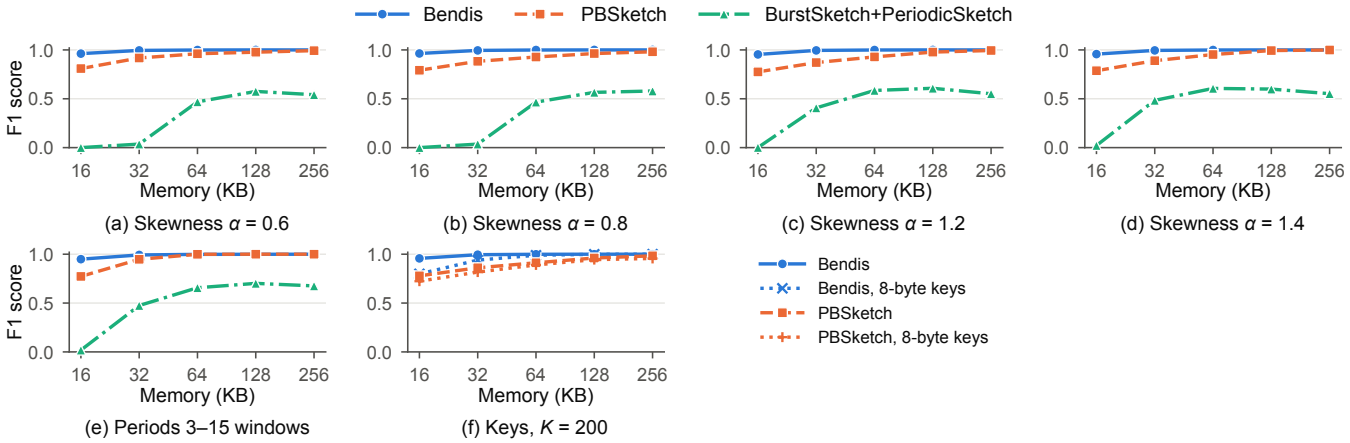


Figure S4: F1 score versus memory on the synthetic streams with skewness $\alpha = 0.6$, 0.8, 1.2, and 1.4 and with periods of 3–15 windows, and with 16-bit fingerprints versus 8-byte item keys in the entries ($\alpha = 1.0$; $K = 200$).

Table S3: F1 score of the CPU implementation (CPU) and of the behavioral model of the P4 program with the CRC hashes of the switch (HW), whose reports the FPGA design reproduces, for every K and w (count windows of 2^{15} arrivals; one run each).

K	w	CAIDA 2016		CAIDA 2018		CAIDA 2019	
		CPU	HW	CPU	HW	CPU	HW
20	2^9	0.842	0.769	0.769	0.850	0.974	0.974
	2^{10}	0.872	0.872	0.872	0.900	0.974	1.000
	2^{11}	1.000	0.974	0.923	0.923	1.000	1.000
	2^{12}	0.974	0.974	0.950	0.974	1.000	1.000
	2^{13}	0.974	1.000	0.974	1.000	1.000	1.000
100	2^9	0.812	0.794	0.912	0.913	0.909	0.899
	2^{10}	0.916	0.928	0.965	0.975	0.947	0.943
	2^{11}	0.980	0.985	0.990	0.975	0.981	0.976
	2^{12}	0.985	0.985	0.995	0.975	0.990	0.990
	2^{13}	1.000	0.990	1.000	1.000	0.995	0.995
200	2^9	0.700	0.726	0.906	0.901	0.849	0.841
	2^{10}	0.800	0.888	0.948	0.943	0.943	0.940
	2^{11}	0.947	0.957	0.979	0.955	0.981	0.986
	2^{12}	0.981	0.986	0.948	0.948	0.995	0.998
	2^{13}	0.994	0.996	0.998	0.998	1.000	1.000
500	2^9	0.636	0.645	0.854	0.850	0.834	0.870
	2^{10}	0.727	0.725	0.944	0.943	0.933	0.948
	2^{11}	0.943	0.951	0.980	0.983	0.980	0.975
	2^{12}	0.973	0.974	0.993	0.991	0.990	0.990
	2^{13}	0.989	0.989	0.997	0.998	0.995	0.998
1,000	2^9	–	–	0.809	0.809	0.825	0.821
	2^{10}	–	–	0.916	0.923	0.937	0.935
	2^{11}	–	–	0.962	0.961	0.980	0.976
	2^{12}	–	–	0.979	0.977	0.993	0.993
	2^{13}	–	–	0.984	0.983	0.996	0.997

Table S4: Control-plane load of BENDIS on Tofino: digests over the whole trace, digests per second, most digests in a window of 32,768 arrivals, and distinct pairs of the exact aggregation.

Trace	w	Digests	Per second	Peak/window	Pairs
CAIDA 2016	2^9	7,548	155	23	2,758
	2^{10}	6,938	143	19	2,511
	2^{11}	6,546	135	19	2,373
	2^{12}	6,451	133	19	2,362
	2^{13}	6,429	132	19	2,357
CAIDA 2018	2^9	12,990	262	30	3,900
	2^{10}	13,383	270	32	3,976
	2^{11}	13,377	270	32	3,973
	2^{12}	13,378	270	31	3,964
	2^{13}	13,387	270	32	3,978
CAIDA 2019	2^9	19,688	397	38	6,000
	2^{10}	20,521	414	36	6,034
	2^{11}	20,484	414	37	6,018
	2^{12}	20,417	412	36	6,009
	2^{13}	20,405	412	36	6,013